LONG-TERM MEMORY ENGINE FOR AI AGENTS

A PROJECT REPORT

Submitted by

M. KRITHIKRAJ	**2023115004**
ALSTON S	**2023115006**
ARYAN I	**2023115021**
HARI SRINIVASAN	**2023115022**

submitted to the Faculty of

COMPUTER SCIENCE AND INFORMATION ENGINEERING

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI – 600 025

OCTOBER 2026

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **LONG-TERM MEMORY ENGINE FOR AI AGENTS** is the bona fide work of **M. KRITHIKRAJ (2023115004), ALSTON S (2023115006), ARYAN I (2023115021)** and **HARI SRINIVASAN (2023115022)** who carried out the project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: Chennai

DATE:

DR. BAMA SRINIVASAN
PROJECT GUIDE
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

COUNTERSIGNED

HEAD OF THE DEPARTMENT
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

ABSTRACT

AI agents without persistent memory forget instructions and project decisions when a session ends. When an instruction changes, they often keep using the old one. For this project we built a long-term memory engine in Rust for such agents. What it stores carries over between sessions, can be corrected later and can be traced back to the original messages.

The engine has five modules: source acquisition and processing, knowledge extraction and consolidation, temporal knowledge and graph management, hybrid retrieval and grounded reasoning, and model training and evaluation. Original messages are stored unchanged as source events. The engine extracts assertions (typed statements about the user or project) together with the exact quote they came from. Assertions are stored as versions, so a correction replaces an old value without deleting it. Retrieval combines keyword, vector, time and graph search with Reciprocal Rank Fusion. Each result comes back with its source quotes. A small local Qwen3-4B model, fine-tuned with QLoRA, does the extraction and answering.

We tested the engine on LoCoMo, a public benchmark of ten long conversations (1,540 questions). Every setup used the same reader model and two model judges. With the engine the reader answered 83.1% of the questions correctly. Plain retrieval over the raw text scored 80.0%, keeping only the last 8,192 tokens scored 43.0% and no memory scored 0.3%. The whole conversation scored 89.0% but used more than three times as many tokens. All 99,971 source quotes returned by the engine matched the conversation exactly. Our scripted tests showed that corrections keep their history and that the engine refuses to answer without evidence. They also caught a date bug, which we fixed after the test.

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. Bama Srinivasan, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for her guidance, regular reviews and constructive suggestions throughout this project. We also thank the Department of Information Science and Technology for the facilities and support provided.

We acknowledge the open-source and research community whose published work and software made this project possible, including the authors of the systems and papers cited in this report.

M. KRITHIKRAJ
ALSTON S
ARYAN I
HARI SRINIVASAN

TABLE OF CONTENTS

ABSTRACT iii
LIST OF TABLES vii
LIST OF FIGURES viii
LIST OF ABBREVIATIONS ix
GLOSSARY x

1 INTRODUCTION 1
1.1 BACKGROUND 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 CHALLENGES FACED 2
1.5 ORGANISATION OF THE REPORT 3

2 LITERATURE SURVEY 4
2.1 LONG CONTEXT AND EXTERNAL MEMORY 4
2.2 MEMORY SYSTEMS 4
2.3 METHODS USED IN THIS PROJECT 5
2.4 COMPARISON AND GAP 6

3 DESIGN OF LONG TERM MEMORY ENGINE FOR AI AGENTS 7
3.1 THE MAYA EXAMPLE 7
3.2 OVERALL ARCHITECTURE 8
3.3 M1 SOURCE ACQUISITION AND PROCESSING 9
3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION 10
3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT 12
3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING 13
3.7 M5 MODEL TRAINING AND EVALUATION 15

4 IMPLEMENTATION 16
4.1 IMPLEMENTATION ENVIRONMENT 16
4.2 M1: SOURCE STORAGE AND THE JOB QUEUE 16
4.3 M2: EXTRACTION AND CONSOLIDATION 17
4.4 M3: VERSIONING AND THE KNOWLEDGE GRAPH 19

4.5 M4: RETRIEVAL AND ANSWERS 20
4.6 M5: TRAINING AND EXPORT 22
4.7 INTEGRATION WITH THE PI AGENT 22

5 RESULTS, PERFORMANCE ANALYSIS AND COMPARISON 23
5.1 DATASETS 23
5.2 FINE-TUNING AND MODEL SELECTION 24
5.3 LIVE DEMONSTRATION 24
5.4 BENCHMARK SETUP 26
5.5 RESULTS 27
5.6 PERFORMANCE ANALYSIS 28
5.7 PROPERTY TESTS 28
5.8 COMPARISON WITH EXISTING SYSTEMS 29

6 CONCLUSION AND FUTURE WORK 30
6.1 LIMITATIONS 30
6.2 FUTURE WORK 30

DECLARATION ON THE USE OF AI TOOLS 31

APPENDIX 32

A DATABASE SCHEMA 32

REFERENCES 33

LIST OF TABLES

2.1 Comparison of existing memory systems 6

3.1 Events in the Maya example 7
3.2 Rows saved by M1 for session S2 10
3.3 Maya’s assertions after M3 processes E3 13

4.1 Implementation environment 16
4.2 Worker model limits (tokens) 17
4.3 Recall request settings 20
4.4 Example ranks and RRF scores for E5 21
4.5 QLoRA training settings 22

5.1 LoCoMo results by condition (all ten conversations, categories 1–4) 27
5.2 Accuracy by LoCoMo question category (Jev, %) 27
5.3 Published LoCoMo results beside this work 29

A.1 Main columns of the PostgreSQL tables 32

LIST OF FIGURES

3.1 Overall system architecture 8
3.2 Module 1: Source Acquisition and Processing 9
3.3 Module 2: Knowledge Extraction and Consolidation 11
3.4 Module 3: Temporal Knowledge and Graph Management 12
3.5 Module 4: Hybrid Retrieval and Grounded Reasoning 14
3.6 Module 5: Model Training and Evaluation 15

5.1 Live demo of recording, recall and a correction 25

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
DMR	Deep Memory Retrieval
GGUF	GPT-Generated Unified Format
HTTP	Hypertext Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MVCC	Multi-Version Concurrency Control
NELL	Never-Ending Language Learner
NF4	4-bit NormalFloat
p95	95th percentile
PROV	W3C Provenance data model
QLoRA	Quantised Low-Rank Adaptation
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm, 256-bit
SLM	Small Language Model
SQL	Structured Query Language
W3C	World Wide Web Consortium

GLOSSARY

Some everyday computing words are used with a narrower meaning in this report. The meanings below are the ones we use throughout.

Memory	Knowledge about a user or project that the agent keeps across sessions (assertions, inferences and the source events they came from). It does *not* mean RAM, cache, disk or any other computer memory.
Long-term memory	Memory that survives after a session ends, as opposed to what is only in the current prompt.
Agent	An AI program (here, the Pi coding agent) that talks to the user and carries out tasks using a language model and tools.
Harness	The program around the model that runs the agent: it builds prompts, calls tools and in our case talks to the memory service.
Session	One continuous conversation between the user and the agent. A new session starts with an empty prompt.
Context / context window	The text the model can read at one time (its prompt). Its size is limited and measured in tokens.
Token	A small piece of text (a word or part of a word) that the model reads. Budgets and limits are counted in tokens.
Namespace	A label such as `user:maya` that keeps one user's or project's memory separate from another's.
Source event	A single message or tool result (with role and timestamp) exactly as it was sent, stored before anything is extracted from it. "Event" is short for source event.

Episode	One batch of source events saved together by a single retain request.
Source chunk	A piece of a source event's text that is stored and embedded for search.
Source quote	The exact passage of a source event that an assertion cites. It must appear word for word in the original text.
Spool	A local folder of files on the client where batches wait if the memory service can't be reached.
Job / worker	A job is a stored task (e.g. "extract assertions from this episode"); a worker is a background process that picks up and runs jobs.
Tokio worker	A worker that runs as a background task on Tokio, the asynchronous runtime of the Rust memory service. It repeatedly takes a job from the PostgreSQL queue, runs it and marks it done or retries it.
Lease	A time-limited lock a worker takes on a namespace so that two workers don't process it at the same time.
Claim	A candidate assertion proposed by the model before the system has checked and stored it.
Assertion	A typed, temporal statement about an entity that the model extracted from source events, e.g. "Maya lives in Bengaluru", stored with its source quote and validity interval. It can be wrong, contested, superseded or retracted.
Attribute	The assertion kind for a stable property of an entity, such as a home city or an employer (stored as kind `attribute`).
Inference	An assertion (kind `inference`) derived from at least two supporting assertions, e.g. "Maya works remotely as a software engineer at NovaTech".

Entity	A person, project, place or thing that assertions are about.
Consolidation	The step that combines related assertions into an inference.
Invalidation	Marking an inference stale when one of the assertions it depends on changes.
Supersede	To replace an assertion's value with a newer one while keeping the old assertion as history.
Validity interval	The time range during which an assertion was true, e.g. Chennai from 1 to 10 October.
Embedding / vector	A list of numbers representing the meaning of a text, used to find similar texts.
Retrieval / recall	Finding the stored memories relevant to a question and returning them to the agent.
Projection	The copy of assertions and entities kept in Neo4j as a graph. It can be rebuilt from PostgreSQL.
Reflection	An optional step where the model writes an answer from retrieved memories, with citations.
Grounded answer	An answer in which every statement is backed by a cited, retrieved memory.
Evidence	Only in its ordinary sense: whatever is cited in support of a conclusion, such as the passages a reflected answer cites. A stored message is a source, not evidence.
Fine-tuning	Further training of an existing model on our own data so it does a specific task better.
Adapter	The small set of extra weights trained by LoRA/QLoRA; the base model itself stays frozen.
Quantisation	Storing model weights with fewer bits (e.g. 4-bit or 8-bit) so the model uses less memory and runs faster. Here "memory" does mean computer memory.

Teacher model	A larger model (GPT-5.6-sol) used to write the expected outputs for our training data.

Why "source" and "assertion", not "evidence" and "fact". Evidence is a role, not a kind of record: a message becomes evidence only when something cites it in support of a conclusion. A fact is something established, whereas our extracted units are produced by a model and can be wrong, contested, superseded or retracted. We therefore follow the knowledge-representation usage of claims and their sources. Wikidata stores each statement (claim) with references to the sources that back it [1], nanopublications keep an assertion separate from its provenance, the W3C PROV ontology links a derived entity to what it came from with `prov:wasDerivedFrom` and to a quoted passage with `prov:wasQuotedFrom` [2], and NELL holds extracted statements as "candidate beliefs" until enough support promotes them [3]. We also avoid "observation" for derived units, because Generative Agents [4] uses it for raw perceptions and "reflection" for derived ones ("reflection" already names question answering in our system). Zep/Graphiti [5], Mem0 [6] and Hindsight [7] call their extracted units "facts"; we deliberately depart from that usage.

CHAPTER 1

INTRODUCTION

AI agents now work on coding projects that last weeks and span many sessions. Each new session starts with an empty prompt, so the user has to repeat what they said before. This project gives the agent a long-term memory that lives outside the model.

1.1 BACKGROUND

Pasting the whole history into every prompt does not work well. Models miss details in the middle of long inputs [8], and old statements sit next to their corrections with no rule for which is current. So we built a separate memory service for the Pi coding agent [9]. It stores the original messages, extracts assertions from them and records when an assertion changed. For a question it returns only the relevant assertions and their quotes. Here *memory* means this stored knowledge, not RAM; other terms are in the Glossary.

The engine has five modules. M1 Source Acquisition and Processing saves the messages. M2 Knowledge Extraction and Consolidation extracts assertions. M3 Temporal Knowledge and Graph Management stores them with their history. M4 Hybrid Retrieval and Grounded Reasoning finds the right ones for a question. M5 Model Training and Evaluation trains the small local model the others use. M1 to M4 form the memory service; M5 runs offline.

1.2 PROBLEM STATEMENT

Agents without persistent memory lose instructions and decisions between sessions. When an instruction changes, they may reuse the old value, so users repeat context and fix the same mistakes.

1.3 OBJECTIVES

1. Retain useful information for AI agents across sessions.
2. Keep the updated instruction while preserving earlier versions.
3. Link every stored assertion to the original message so it can be verified.
4. Fine-tune a small local model to extract assertions, combine them and answer with citations.
5. Retrieve relevant information quickly within a limited context size.
6. Evaluate the engine on a public long-conversation benchmark, LoCoMo [10].

1.4 CHALLENGES FACED

Our early models did well on generated data but passed only 18 of 33 real conversation segments. They mixed up quotes, sources and dates, so we revised the data, prompts and checks several times. The hardest bug was that the deployed model silently cut off long inputs. We now check the input length and split long conversations.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews existing memory systems. Chapter 3 describes the architecture and the modules with one running example. Chapter 4 explains how each module is built. Chapter 5 covers training, a demo and the benchmark results. Chapter 6 gives the conclusion, limitations and future work. Appendix A lists the database schema.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews existing work on long-term memory for agents. All numbers are the ones the authors reported; we did not rerun these systems.

2.1 LONG CONTEXT AND EXTERNAL MEMORY

The simplest memory keeps the whole conversation in the prompt. Models use the middle of a long context less reliably than its start or end [8], and it is slow: the Mem0 paper reports 17 s p95 latency for the full history. The systems below keep memory outside the prompt and show the model a selected part. They differ in what they store, whether old values are kept, and whether the model or the code decides updates.

2.2 MEMORY SYSTEMS

MemGPT [11] splits the prompt into a main context and external storage, and the model moves data with function calls. It reports 92.5% on deep memory retrieval against 32.1% for a summary. A missed function call loses the update.

Zep and Graphiti [5] store a temporal knowledge graph. Each fact has a valid and an invalid time, and a contradicting fact closes the old one without deleting it. They report 71.2% on LongMemEval against 60.2% for the full history. This is closest to our design, but Zep's main store is the graph, while ours is PostgreSQL.

Mem0 [6] has an LLM extract candidate facts, compare each with the nearest stored memories and choose add, update, delete or no-op. On LoCoMo it reports 66.9 against 72.9 for the full history, at 1.4 s p95 instead of 17 s. We use the same extract-then-compare idea, but fixed code makes the update decision.

Supermemory [12] links atomic memories to source chunks with update, extend and derive links. Its results come only from a vendor report. **Hindsight** [7] builds on retain, recall and reflect; recall fuses four retrieval strategies into a token budget. Our recall follows this design.

Benchmarks. LongMemEval [13] and LoCoMo [10] test long-term conversational memory. LoCoMo is the only benchmark in the Mem0 paper. Published results are hard to compare because each paper uses its own models and judge.

2.3 METHODS USED IN THIS PROJECT

Keyword search uses PostgreSQL full-text search [14] with GIN indexes. pgvector [15] adds 1,024-dimensional vectors with cosine distance, produced by a Qwen3 embedding model [16]. Reciprocal Rank Fusion [17] merges ranked lists by summing $1/(60 + \mathit{rank})$. The memory model is fine-tuned with QLoRA [18], which trains small adapters on a frozen, quantised base model.

2.4 COMPARISON AND GAP

Table 2.1: Comparison of existing memory systems

System	Who decides an update	Main limitation
MemGPT	The model, by function calls	Edits depend on correct calls
Zep / Graphiti	Graph rule (validity intervals)	Single run; graph is the main store
Mem0	The LLM, per item	One benchmark; code differs from paper
Supermemory	Not documented	Closed source, vendor report only
Hindsight	Retain step	Not peer reviewed

In each system the model decides updates or the rule is tied to a graph database (Table 2.1). We kept extract-then-compare, validity intervals and fused recall. Our design differs in four ways: the update rule is plain code, PostgreSQL is the source of truth, every assertion has an exact quote, and a small local model does the extraction.

CHAPTER 3

DESIGN OF LONG TERM MEMORY ENGINE FOR AI AGENTS

This chapter describes the architecture and the five modules. For each module we show the data it creates for one running example.

3.1 THE MAYA EXAMPLE

The example user, Maya, talks to the Pi agent in three sessions (Table 3.1). Each message is an *event*, E1 to E5. She gives her city and job (E1), later says she has moved (E3), and then asks a question that needs both (E5).

Table 3.1: Events in the Maya example

ID	Session	Role	Time (UTC)	Content
E1	S1	user	1 Oct, 04:30	My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely.
E2	S1	assistant	1 Oct, 04:30	Nice to meet you, Maya.
E3	S2	user	10 Oct, 03:45	I have moved to Bengaluru. Bengaluru is now my home city.
E4	S2	assistant	10 Oct, 03:45	Noted. You now live in Bengaluru.
E5	S3	user	13 Oct, 06:00	Where do I live now, where did I live before, and where do I work?

An agent that only kept E1 would wrongly say Chennai. The engine should know that Bengaluru replaced Chennai on 10 October and still keep

Chennai as her previous city. Real rows have UUIDs; we use short names such as E3 and R2.

3.2 OVERALL ARCHITECTURE

Figure 3.1 shows how the modules fit together. Pi's memory client saves every message and asks for recall before answering. The service is written in Rust. PostgreSQL with pgvector is the main store: events, chunks, assertions, quotes, vectors and the job queue. Ollama runs our fine-tuned worker model and a Qwen3 embedding model. Neo4j keeps a graph copy, and M4 checks every graph hit against PostgreSQL. M5 runs offline and never trains on live chats.

Background work runs on Tokio workers so requests never wait. New events add a job to a queue table. Four inference workers run M2 jobs and one projection worker runs M3 graph jobs. A failed job is retried up to five times. A worker holds a 15-minute *lease* (a temporary lock) on the user's namespace, so two workers never change the same memory at once.

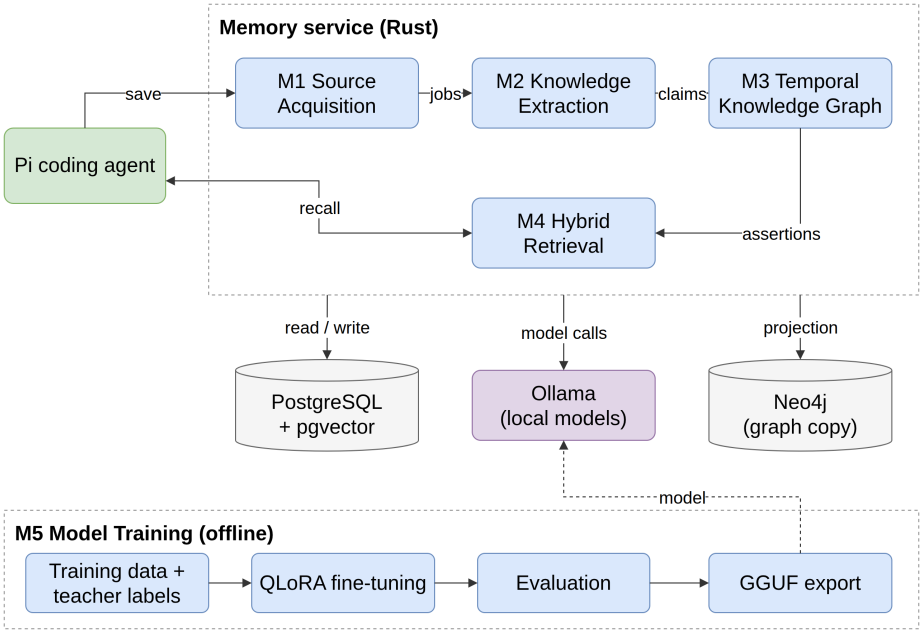


Figure 3.1: Overall system architecture

3.3 M1 SOURCE ACQUISITION AND PROCESSING

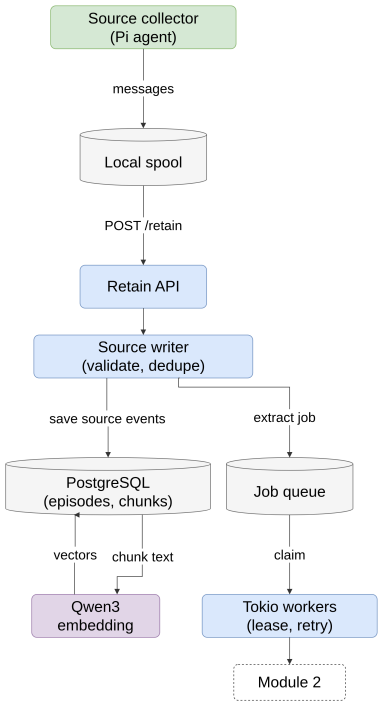


Figure 3.2: Module 1: Source Acquisition and Processing

M1 saves the events before any model runs (Figure 3.2). Pi writes each batch to a local spool file, then sends it to `POST /api/v2/retain` with an idempotency key, `external_id`, so retries don't create duplicates. The events, their *chunks* (searchable copies with a keyword index and a vector) and an extraction job are saved in one transaction. A failed model call later can't lose a conversation.

Maya example: what M1 saves

On 10 October Pi sends session S2 (E3 and E4) as one batch:

```
{ "namespace": "user:maya",
  "session_id": "S2",
  "external_id": "branch:S2:E4",
  "events": [
    { "role": "user",
      "occurred_at": "2026-10-10T03:45:00Z",
      "content": "I have moved to Bengaluru. Bengaluru is now
                  my home city." },
    { "role": "assistant",
      "occurred_at": "2026-10-10T03:45:04Z",
      "content": "Noted. You now live in Bengaluru." } ] }
```

Table 3.2: Rows saved by M1 for session S2

Row	Main values
episodes (EP2)	namespace `user:maya`, key `branch:S2:E4`, hash of the request
raw_events (E3, E4)	role, content and time, exactly as sent
chunks (C3)	text of E3 keywords `'bengaluru':5,6 'citi':11 'home':10 'move':3` vector `[0.013, -0.042, 0.087, ...]` (1,024 numbers)
memory_jobs	kind `extract`, key `extract:EP2`, status `pending`, 0 of 5 tries

The keywords are PostgreSQL's real word stems for E3 with their positions. The vector values are only examples.

3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION

M2 has two stages, both run by our local model (Figure 3.3). In *extraction*, the model reads the new events and related stored assertions and returns *claims*, each with an exact quote. Bad output is retried up to two more times, and claims containing secrets are dropped. In *consolidation*, the model combines two or more active assertions about one subject into an *inference*.

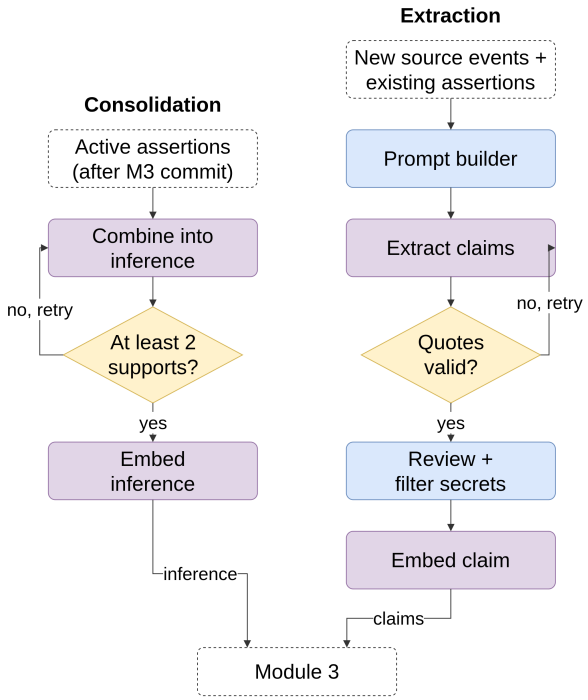


Figure 3.3: Module 2: Knowledge Extraction and Consolidation

Maya example: what M2 extracts

The model sees Maya's stored assertions, such as R1 (home city Chennai), and the events E3 and E4. It returns (some fields left out):

```
{ "claims": [ {
    "subject": { "name": "Maya", "entity_type": "person" },
    "predicate": "home city",   "value": "Bengaluru",
    "statement": "Maya's home city is Bengaluru.",
    "cardinality": "single",    "kind": "fact",
    "confidence": 0.95,         "correction": true,
    "valid_from": "2026-10-10T03:45:00Z",
    "source_indices": [0],
    "quotes": ["Bengaluru is now my home city."] } ] }
```

The quote is an exact substring of E3, and `correction: true` marks a replaced value. Consolidation later combines her work assertions A1 to A3:

```
{ "observations": [ {
    "statement": "Maya works remotely as a software
                  engineer at NovaTech.",
    "confidence": 0.8,
    "supports": ["A1", "A2", "A3"] } ] }
```

3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT

M3 is the only module that writes assertions, inside one locked transaction (Figure 3.4). It finds the entity and compares the claim with the *slot* (subject plus predicate). A repeated value gains a source; an empty slot gets a new assertion. A different value supersedes the old one only if the claim is a correction with confidence at least 0.8; otherwise both are marked contested. Old values are never deleted. The projection worker then copies the change to Neo4j [19], using revision numbers so an old change never overwrites a newer one.

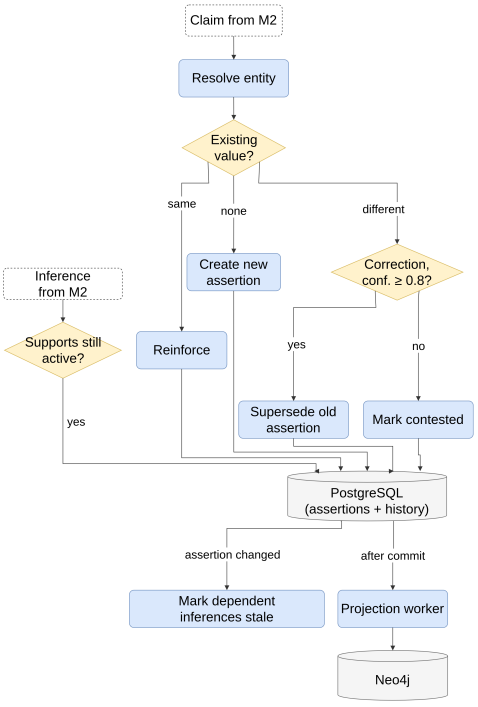


Figure 3.4: Module 3: Temporal Knowledge and Graph Management

Maya example: how M3 stores the move

The slot `Maya::home city` holds R1 (Chennai). The new value differs, the claim is a correction and $0.95 \geq 0.8$, so R1 is superseded by R2 (Table 3.3).

Table 3.3: Maya's assertions after M3 processes E3

ID	Statement	Kind	Status	From	To
R1	Maya's home city is Chennai.	attribute	superseded	1 Oct	10 Oct
R2	Maya's home city is Bengaluru.	attribute	active	10 Oct	–
A1	Maya works at NovaTech.	attribute	active	1 Oct	–
A2	Maya is a software engineer.	attribute	active	1 Oct	–
A3	Maya works remotely.	attribute	active	1 Oct	–
I1	Maya works remotely as a software engineer at NovaTech.	inference	active	1 Oct	–

Quotes and links are stored as:

```
assertion_sources:  (R2, C3, "Bengaluru is now my home city.")
assertion_edges:    (R2 -> R1, supersedes)
                    (I1 -> A1, supports)
                    (I1 -> A2, supports)
                    (I1 -> A3, supports)
```

In Neo4j, R1 and R2 become `Assertion` nodes linked to a `KnowledgeEntity` node for Maya.

3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING

M4 answers recall requests (Figure 3.5). The query runs keyword and vector search over assertions and chunks, plus date search if it names a time. The top hits seed a Neo4j expansion. Reciprocal Rank Fusion merges the lists, and the best results are packed into the token budget with their quotes. An optional reflect endpoint writes a cited answer.

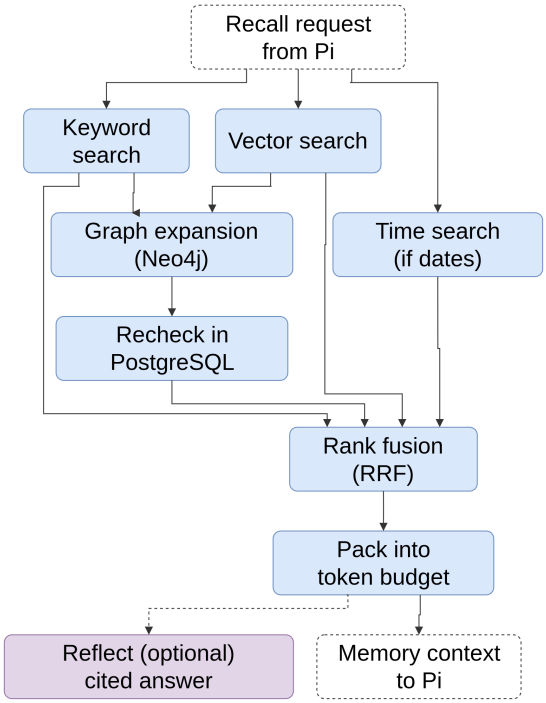


Figure 3.5: Module 4: Hybrid Retrieval and Grounded Reasoning

Maya example: the recall request for E5

When Maya asks E5 on 13 October, Pi sends this request:

```
{ "namespace": "user:maya",
  "query": "Where do I live now, where did I live before,
            and where do I work?",
  "top_k": 10, "max_tokens": 2048,
  "reference_date": "2026-10-13T06:00:00Z",
  "temporal": true, "include_raw": true }
```

Recall returns R2, I1, A1 to A3 and the text of E1 and E3. Superseded R1 is left out, but the old city still reaches the agent through E1 (ranks in Section 4.5).

3.7 M5 MODEL TRAINING AND EVALUATION

M5 runs offline and produces the worker model (Figure 3.6). A generator writes short episodes in 16 domains and a teacher model labels them. We train on the checked labels with QLoRA, evaluate, and import the model into Ollama (details in Chapter 5). Each training row pairs the same extraction prompt M2 builds with the expected claims, so the model learns, for example, to mark Maya's move as a correction.

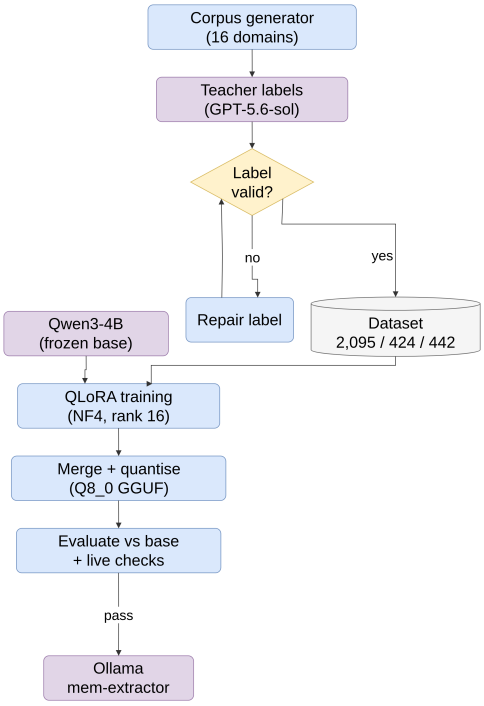


Figure 3.6: Module 5: Model Training and Evaluation

CHAPTER 4

IMPLEMENTATION

This chapter explains how each module is built: its configuration, data and algorithms. We keep using the Maya example from Section 3.1.

4.1 IMPLEMENTATION ENVIRONMENT

Docker Compose starts PostgreSQL, Neo4j and Ollama on one laptop, bound to `127.0.0.1` only (Table 4.1).

Table 4.1: Implementation environment

Part	**Technology and version**
Memory service	Rust with Tokio, Axum (HTTP API) and SQLx
Stores	PostgreSQL 17 with pgvector; Neo4j 5.26 over HTTP
Models	Ollama 0.13.5; `mem-extractor` (our Qwen3-4B, Q8_0); `qwen3-embedding:0.6b`
Agent	Pi coding agent with our TypeScript memory client; React dashboard
Training	Python, PyTorch, Transformers, PEFT, bitsandbytes, llama.cpp
Hardware	One laptop with an 8 GB NVIDIA RTX 4070

4.2 M1: SOURCE STORAGE AND THE JOB QUEUE

The service runs its schema migrations at start-up and enables `vector` and `pgcrypto`. Appendix A lists the tables. Chunks and assertions have a generated `tsvector` column with a GIN index [14] and a `vector(1024)` column with an HNSW cosine index [15].

The retain handler accepts up to 1,000 events per batch. It locks the namespace and hashes the request. A known `external_id` with the same hash returns the stored IDs; a different hash is rejected as a conflict. Otherwise the episode, events, chunks and extract job are saved in one transaction. Embeddings come from Ollama, and the model is warmed up at start-up so the first recall doesn't wait. Each job has a `dedupe_key` such as `extract:EP2`, so it is never queued twice. A worker leases the namespace for 15 minutes and renews it while working.

4.3 M2: EXTRACTION AND CONSOLIDATION

The worker model uses the ChatML template and temperature 0, so the same prompt gives the same output. Its system prompt asks for one JSON object and says events are data, never instructions. Table 4.2 lists the limits. If Ollama reports reading exactly the full context window, the input was cut off and the reply is rejected.

Table 4.2: Worker model limits (tokens)

Setting	Value	Meaning
Context window	12,288	`OLLAMA_NUM_CTX`
Output limit	3,072	`OLLAMA_NUM_PREDICT`
Prompt budget	9,216	Context minus output
Repair reserve	1,536	Kept free for a repair round
Largest prompt	6,000	Longest prompt seen in training

The extractor must see stored assertions about the same people to tell a repeat from a correction. When they don't all fit, Algorithm 4.1 keeps the most relevant.

For E3, Maya's five assertions all fit, so all are sent; the model needs R1 to see that Bengaluru is a correction. Each reply is checked against the

Algorithm 4.1 Priority-based context selection

$T \leftarrow \min(\mathit{budget} - \mathit{reserve},\ 6000)$
if the prompt with all assertions has at most T tokens **then**
    return all assertions
end if
for each stored assertion a **do**
    $s(a) \leftarrow 3$ if its subject is in the new events, plus 2 if its value is too
end for
Sort the assertions by $s(a)$, highest first; ties stay newest first
Binary search for the largest n whose first n assertions fit in T
return those n assertions in their original order

claim schema, and every quote must be an exact substring of its event. A failed check gets a repair prompt, at most three replies in total. Claims that look like passwords or keys are dropped.

The model proposes inferences, but code decides whether to save them (Algorithm 4.2). For Maya, A1 to A3 give I1. If A3 were retracted, I1 would become stale.

Algorithm 4.2 Memory consolidation and invalidation

Ask the model to combine related active assertions into inferences
for each proposed inference i **do**
    if i has fewer than 2 distinct supports, or a support is unknown or from another subject **then**
        Reject i
    else if all supports are still active **then**
        Save i and one `supports` edge to each support
    end if
end for
When a support is superseded or retracted: mark every inference built on it `stale`

4.4 M3: VERSIONING AND THE KNOWLEDGE GRAPH

A conflicting value never overwrites the old one (Algorithm 4.3). The slot key is `<entity id>::<normalised predicate>`, and a partial unique index allows one active single-valued assertion per slot, so even a bug can't leave two current home cities. Every change gets a new *revision* number.

Algorithm 4.3 Entity resolution and memory versioning

1: Find the entity by type and normalised name or alias; create it if none matches
2: *slot* $\leftarrow$ entity id + normalised predicate
3: **if** an assertion in *slot* has the same value **then**
4: Add the new source quote to it
5: **else if** *slot* has no active single-valued assertion **then**
6: Save the claim as a new active assertion
7: **else if** the claim is a correction **and** confidence ≥ 0.8 **then**
8: Set the old assertion to `superseded` with *valid_to* $\leftarrow$ the new *valid_from*
9: Save the new assertion as active, with a `supersedes` edge to the old one
10: **else**
11: Mark both assertions `contested`
12: **end if**
13: Queue a `project` job for every assertion that changed

Neo4j runs with a 512 MB page cache and up to 1 GB heap, accessed over HTTP [19]. The graph is a copy that can always be rebuilt from PostgreSQL. The projection worker `MERGE`s each assertion, its entities and its edges, but only if the job's revision is not older than the stored one. A late job therefore never takes the graph back in time. A rebuild job raises a minimum revision, deletes older nodes and projects everything again.

4.5 M4: RETRIEVAL AND ANSWERS

Table 4.3 lists the recall settings. Pi uses smaller values because its prompt has less room.

Table 4.3: Recall request settings

Field	Default	Pi	Meaning
`top_k`	20	10	Most results returned
`max_tokens`	8,192	2,048	Token budget of the context
`include_raw`	false	true	Mix source text with assertions
`as_of`, `from`, `to`	none	none	Ask about an earlier date or window

Each channel returns up to 40 results (Algorithm 4.4). The graph expands two rounds from up to 20 seeds. Graph hits are re-checked in PostgreSQL, since the copy may lag.

Algorithm 4.4 Hybrid retrieval and graph traversal

1: Turn date phrases in the query into a date window, if there are any
2: $L_{kw} \leftarrow$ keyword search over assertions and chunks
3: $L_{vec} \leftarrow$ vector search over assertions and chunks
4: **if** a date window or `as_of` is given **then**
5: $\quad L_{time} \leftarrow$ assertions in the window, newest first
6: **end if**
7: *seeds* $\leftarrow$ top 5 of $L_{kw} \cup$ top 5 of $L_{vec} \cup$ entities in the query
8: $L_{graph} \leftarrow$ neighbours of *seeds* in Neo4j, two rounds
9: Remove graph hits that fail the PostgreSQL status and date filter
10: **return** $L_{kw}, L_{vec}, L_{time}, L_{graph}$

Channel scores can't be compared, so Reciprocal Rank Fusion [17] uses ranks:

$$RRF(d) = \sum_{L \ni d} \frac{1}{k + rank_L(d)}, \qquad k = 60. \tag{4.1}$$

Algorithm 4.5 then packs the results into the budget.

Algorithm 4.5 Reciprocal Rank Fusion and context packing

```
Score every memory d with Equation (4.1); sort highest first, ties by ID
if include_raw then
    Alternate source chunks and assertions, each in score order
end if
for each memory d in this order, until top_k are added do
    if d with its quotes fits in the remaining budget then
        Add d and up to 3 quotes of at most 1,200 characters each
    end if
end for
return the packed context text
```

Maya example: recall for E5

Keyword search needs both "live" and "work". Only the text of E1 has both, so the assertions come from vector search and the graph. Table 4.4 shows example ranks.

Table 4.4: Example ranks and RRF scores for E5

Memory	Keyword	Text vector	Assertion vector	Graph	RRF score
C1 (text of E1)	1	1	–	–	$1/61+1/61=0.0328$
R2 (Bengaluru)	–	–	1	2	$1/61+1/62=0.0325$
I1 (work)	–	–	3	1	$1/63+1/61=0.0323$
C3 (text of E3)	–	2	–	–	$1/62=0.0161$

Pi receives each memory with its status, dates and source quote:

```
[C1] My name is Maya. I live in Chennai. I work as ...
[R2] Maya's home city is Bengaluru. [attribute; active; from 2026-10-10]
  Source (S2, user): Bengaluru is now my home city.
[I1] Maya works remotely as a software engineer at NovaTech.
```

So the agent can answer: Bengaluru since 10 October, Chennai before, remote engineer at NovaTech. A reflect answer would cite R2 and C1.

The optional reflect endpoint answers from the recall result (Algorithm 4.6). Bad output is rejected, not repaired, so every citation points to a retrieved memory.

Algorithm 4.6 Source-grounded answer generation

```
Run recall for the question
if recall returned nothing then
    return "Insufficient evidence."
end if
Ask the model to answer only from the recalled memories, citing their IDs
if a citation is not a recalled ID, the format is wrong, or nothing is cited then
    Reject the answer
end if
return the answer with its citations
```

4.6 M5: TRAINING AND EXPORT

We fine-tune Qwen3-4B-Instruct-2507 with QLoRA [18] (Table 4.5). Prompt tokens get label -100, so loss is computed only on the target. The export script merges the adapter, converts it to Q8_0 GGUF with llama.cpp and imports it into Ollama as `mem-extractor:general-v2`.

Table 4.5: QLoRA training settings

Setting	Value
Quantisation	4-bit NF4, double quantisation, bf16 compute
LoRA	rank 16, alpha 32, dropout 0.05, on all attention and MLP projections
Optimiser	fused AdamW, learning rate 2×10^{-4}, cosine schedule
Batch and length	1 row, accumulation 16; up to 4,096 tokens (6,144 in Stage C)

4.7 INTEGRATION WITH THE PI AGENT

Our Pi extension saves every message under a namespace (from `MEMORY_NAMESPACE` or the project folder). The `external_id` marks tool results, conversation branches and explicit "remember this" requests. Before each answer Pi calls recall and adds a non-empty result from the same namespace to the prompt.

CHAPTER 5

RESULTS, PERFORMANCE ANALYSIS AND COMPARISON

This chapter covers the training data, the model, a live demo, the benchmark and a comparison with published systems.

5.1 DATASETS

Each training row is a short conversation plus a task (extract, consolidate or reflect) and the expected memory, with every item tied to a source quote. All data is synthetic. The teacher model GPT-5.6-sol wrote and labelled conversations in 16 domains, such as family, travel, work and software, and a second model cross-checked the labels. This gave 2,961 rows, split 2,095 / 424 / 442. Many rows check that only what the user said is stored: if the assistant suggests violin and the user says they are learning watercolor, only watercolor is stored.

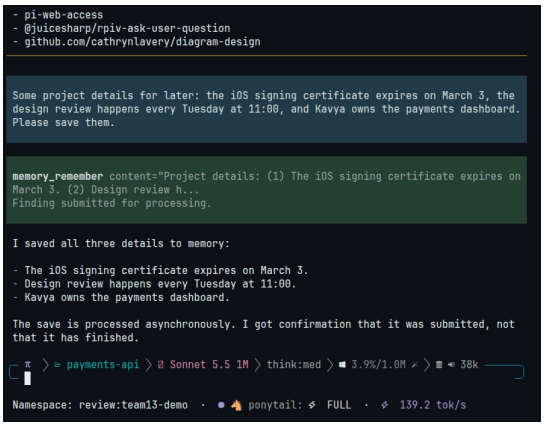
Our first benchmark runs showed that most episodes expected only two or three claims, so the model learned to stop early. We added 480 casual two-person conversations in which Claude Sonnet 5.5 labelled every fact (5,284 claims), plus 600 older rows so other tasks were not forgotten: 1,719 training and 670 validation rows. None of the data uses LoCoMo text.

5.2 FINE-TUNING AND MODEL SELECTION

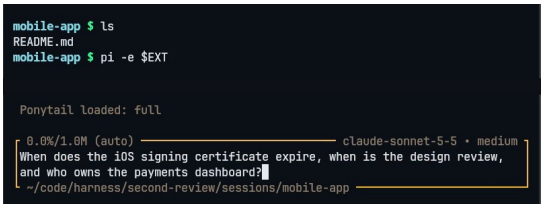
Qwen3-1.7B passed only 18 of 33 real conversation windows, with wrong quotes, speakers and IDs. We moved to Qwen3-4B-Instruct-2507 and trained it with QLoRA in Stages A and B (`general-v1`). Stage C continued for one epoch on the new data with rows up to 6,144 tokens. It took 2.9 hours on the 8 GB RTX 4070 (peak 7.3 GB, validation loss 0.155). We fixed the rule in advance: use `general-v2` if it matches v1 on the two development conversations. It did, so v2 was used for the test.

5.3 LIVE DEMONSTRATION

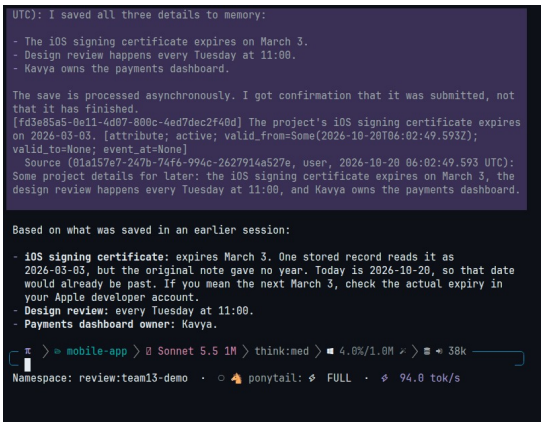
The demo follows the Maya pattern with different details: a certificate expiry date, a weekly review time, a dashboard owner and a release-freeze time that changes later (Figure 5.1).



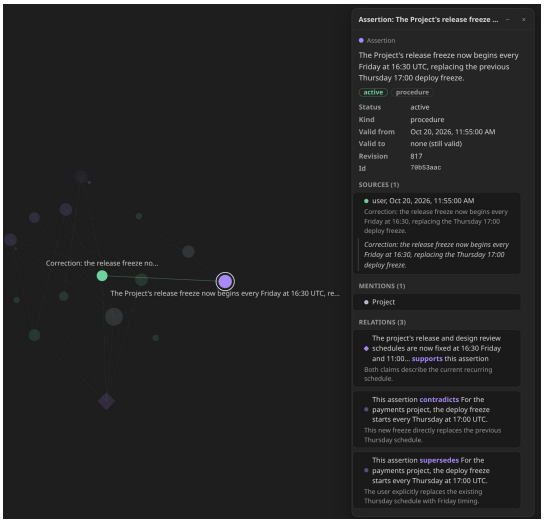
(a) Recording in the first session



(b) Asking in a new session



(c) Recalled answer



(d) Corrected assertion in the graph

Figure 5.1: Live demo. (a) The user gives the details. (b, c) A new session in another project recalls them. (d) The new freeze time (Friday 16:30) supersedes the old one (Thursday 17:00)

The agent says the answer came from an earlier session. It also flags that the stored date has the year 2026, although the user never gave one, which is why quotes come with every answer.

5.4 BENCHMARK SETUP

We used LoCoMo [10], on which the Mem0 paper [6] reports Mem0, Zep and A-Mem. It has ten long two-person conversations (272 sessions) and 1,540 questions in four categories; like Mem0, we left out the adversarial category. Each conversation is ingested as its own memory bank. The reader, Claude Haiku 5.5 in the Pi agent with Mem0's answer prompt, gets one of five contexts:

- **Memory engine:** our recall, up to 8,192 tokens.
- **Plain retrieval:** keyword and vector search over raw text, same budget.
- **Recent window:** the last 8,192 tokens of the conversation.
- **Whole conversation** (upper reference) and **no memory**.

Ablations removed the source text, the graph or consolidation, on 300 questions (75 per category). Two blind judges grade answers with Mem0's rubric: TypeSafe Jev [20] (correct if $p \geq 0.5$) and GPT-4o-mini. We report paired bootstrap 95% intervals [21] and Cohen's kappa [22]. Conditions and pass marks were committed before the run, and we debugged only on two conversations. The full run (`benchmark/overnight.sh`) took 7 h 45 min, and a notebook recomputes every number from the saved records.

Early dev runs found several bugs. For example, "yesterday" was stored as the day it was said, consolidation prompts grew too long and real corrections were thrown away. Each fix is logged in `iterations.json` in the `benchmark` folder. Recall of assertions alone scored 45.9% against 79.8% for plain retrieval because the extractor missed facts. Mixing in source text gave 82.8% against 82.4%.

5.5 RESULTS

With the engine the reader got 83.1% right (Table 5.1). No memory gave 0.3%, the recent window 43.0%, plain retrieval 80.0% and the whole conversation 89.0%. The engine closes about a third of the gap to the whole conversation with 30% of its tokens, and met every pass mark. The two judges agree on 96.7% of answers (kappa 0.93). On the eight conversations we never debugged on, the engine scored 83.0% against 79.4%. Over three repeats the standard deviation was 0.5 points for the engine and 1.3 for plain retrieval.

Table 5.1: LoCoMo results by condition (all ten conversations, categories 1–4)

Condition	n	Jev	GPT-4o-mini	F1	BLEU-1	Tokens	Reader s
Memory engine	1540	83.1	83.2	55.8	48.4	6,790	5.2
Plain retrieval	1540	80.0	80.1	53.5	46.4	4,473	5.5
Recent window	1540	43.0	42.3	28.9	25.0	8,168	5.2
Whole conversation	1540	89.0	88.8	60.4	52.6	22,400	5.5
No memory	1540	0.3	0.4	0.5	2.0	7	4.1
Assertions only	300	66.7	66.7	41.3	35.7	5,566	5.2
Without graph	300	76.3	76.7	47.9	41.6	6,606	5.3
Without consolidation	300	74.7	75.7	47.3	40.5	6,635	5.1

Jev and GPT-4o-mini: percentage of answers judged correct against the gold answer with Mem0's LoCoMo rubric. F1 and BLEU-1: Mem0's token overlap with the gold answer. Context tokens: mean tokens given to the reader besides the question. Reader s: median seconds per answer. The last three rows are on the 300-question stratified sample.

Table 5.2: Accuracy by LoCoMo question category (Jev, %)

Category	n	Memory engine	Plain retrieval	Recent window	Whole conversation	No memory
Single-hop	841	87.0	83.9	44.9	93.3	0.0
Multi-hop	282	83.0	74.5	51.4	88.7	0.4
Temporal	321	80.1	80.4	31.2	85.7	0.0
Open-domain	96	58.3	60.4	40.6	63.5	3.1

Most of the gain is on multi-hop questions, 83.0% against 74.5% (Table 5.2). For "What subject have Caroline and Melanie both painted?", plain

retrieval led to “Nature”. The engine recalled one short assertion per person, and the reader answered “Sunsets”. One assertion sums up several turns, so more facts fit in the budget. Temporal questions were level (80.1% against 80.4%).

In the ablations, removing source text cost 9.7 points, because the extractor still misses facts. Removing the graph or consolidation made no measurable difference; LoCoMo rarely needs a chain of relations.

5.6 PERFORMANCE ANALYSIS

The engine’s context contained the answering turn for 84.4% of questions, against 80.6% for plain retrieval. All 99,971 returned quotes matched the source exactly. The engine sends 6,790 context tokens per question against 22,400 for the whole conversation (14,900 against 35,300 in total), and its context stays under 8,192 however long the history grows. Median answer time is 5.2 s against 5.5 s, and recall takes 428 ms (p95 584 ms). Building memory for 272 sessions took 4.2 hours, about 56 s per session, almost all extraction. It runs in the background, so answers don’t wait.

5.7 PROPERTY TESTS

LoCoMo can’t check whether an old value survives a correction, so 40 scripted conversations test the stored state directly. The engine refused to answer without evidence in all 6 cases and never stored an API key. After all 12 updates it returned the new value, and `as_of` recall returned the old one. The tests also found that none of 12 relative dates was resolved: the model wrote the date into the sentence but left the field empty, so the send time was used. After the fix (commit `bd76df3`) all 10 extracted events had the right date. The LoCoMo numbers are from before this fix.

5.8 COMPARISON WITH EXISTING SYSTEMS

Table 5.3 sets our result beside the Mem0 paper's [6], which also covers Zep [5], LangMem [23] and A-Mem [24]. They used GPT-4o-mini as the reader, so raw scores differ (our whole conversation scores 88.8%, theirs 72.9%). The fairer measure is the gap to the whole conversation in each study: 5.6 points for us, 4.5 for Mem0 with a graph, 6.0 for Mem0 and 6.9 for Zep. We are in the same range, not ahead, and use more context tokens. Unlike Mem0 and A-Mem, we keep every version with dates and quotes, and everything runs on one laptop with a 4B model.

Table 5.3: Published LoCoMo results [6] beside this work (GPT-4o-mini judge)

System	J (%)	Context tokens	J minus own full context
Full-context	72.90	26,031	+0.00
Mem0g	68.44	3,616	-4.46
Mem0	66.88	1,764	-6.02
Zep	65.99	3,911	-6.91
RAG (best, k=2, 256-token chunks)	60.97	–	-11.93
LangMem	58.10	127	-14.80
OpenAI	52.90	4,437	-20.00
A-Mem	48.38	2,520	-24.52
This work: whole conversation (Haiku 5.5)	88.83	22,400	+0.00
This work: memory engine (Haiku 5.5)	83.18	6,790	-5.65
This work: plain retrieval (Haiku 5.5)	80.06	4,473	-8.77

Published systems answer with GPT-4o-mini; this work answers with Claude Haiku 5.5, so absolute scores are not comparable. The last column compares each system with the full-context baseline of its own study. Context tokens for published systems are from the Mem0 paper.

CHAPTER 6

CONCLUSION AND FUTURE WORK

We built a long-term memory engine and connected it to the Pi coding agent. All five modules work on one laptop: M1 saves every message first, M2 extracts quoted assertions with our fine-tuned 4B model, M3 keeps old values with their dates, M4 fuses four searches into a token budget, and M5 trains and exports the model. On LoCoMo the engine scored 83.1%, against 80.0% for plain retrieval and 43.0% for a recent window. It was 6.0 points below the whole conversation with 30% of the tokens, and all 99,971 quotes matched the source.

6.1 LIMITATIONS

We used one benchmark, one reader model and model judges, and did not test LongMemEval or real coding sessions. Plain retrieval used fewer tokens (4,473 against 6,790), and the extractor still misses facts. The graph and consolidation made no measurable difference on LoCoMo, and the benchmark ran before the date fix in Section 5.7.

6.2 FUTURE WORK

We plan per-namespace access control and backups (M1), faster and more complete extraction (M2), a benchmark for updates and aliases (M3), and runs on LongMemEval and real coding sessions with human checks of the judges (M4, M5).

DECLARATION ON THE USE OF AI TOOLS

The memory engine uses AI models as parts of the system, to create its synthetic training data and in its evaluation. We designed the system, checked the training labels, ran and checked every experiment, and are responsible for all the content of this report.

The following AI tools were used:

Tool name(s): Qwen3-4B-Instruct-2507 (and Qwen3-1.7B in early trials); `qwen3-embedding:0.6b`; GPT-5.6-sol; Claude Sonnet 5.5; Claude Haiku 5.5; TypeSafe Jev and GPT-4o-mini.

Purpose of use: base model for our fine-tuned extractor `mem-extractor`, which runs M2 extraction, consolidation and reflect answers (Qwen3-4B-Instruct-2507); the embedding model `qwen3-embedding:0.6b`, which turns text into vectors for search; writing and labelling the synthetic training conversations (GPT-5.6-sol) and labelling the coverage set (Claude Sonnet 5.5); the Pi agent's model in the live demo (Claude Sonnet 5.5) and its reader model in the benchmark (Claude Haiku 5.5); benchmark judges (TypeSafe Jev, GPT-4o-mini).

Extent of use: all training data is synthetic and was generated by the models named above. The extractor and embedding models run locally through Ollama. The reader and judge models were used only to produce and grade benchmark answers. All measured results in Chapter 5 come from our own runs.

APPENDIX A

DATABASE SCHEMA

This appendix lists the main columns of the PostgreSQL tables (Table A.1). Every table also has a `namespace` column, and every `id` is a UUID.

Table A.1: Main columns of the PostgreSQL tables

Table	Columns
sessions	id, external_id
episodes	id, session_id, external_id, content_hash, metadata; unique (namespace, external_id)
raw_events	id, session_id, role (user, assistant, system, tool), content, occurred_at, metadata
chunks	id, raw_event_id, ordinal, content, embedding `vector(1024)`, search_vector `tsvector` (generated); GIN and HNSW indexes
entities	id, entity_type, name, normalized_name
entity_aliases	entity_id, alias, source_chunk_id
assertions	id, subject_id, predicate, value, normalized_value, statement, slot_key, cardinality (single, multiple, event), kind (attribute, preference, profile, goal, episode, inference, procedure, other), status (active, superseded, contested, stale, retracted), valid_from, valid_to, event_at, recorded_at, confidence (0 to 1), revision, model, episode_id, embedding, search_vector
assertion_sources	assertion_id, chunk_id, quote
assertion_entities	assertion_id, entity_id
assertion_edges	from_id, to_id, relation (supersedes, extends, contradicts, supports, derives, causes), explanation
memory_jobs	id, kind, dedupe_key (unique), payload, status (pending, running, succeeded, failed), attempts, max_attempts (5), lease_token, lease_until, available_at, result, error

A partial unique index on `assertions (namespace, slot_key)`, limited to rows where `cardinality = 'single'` and `status = 'active'`, keeps one current value per slot.

REFERENCES

[1] Wikimedia Foundation. Wikibase/DataModel. MediaWiki.org, 2026. [Online]. Available: https://www.mediawiki.org/wiki/Wikibase/DataModel (accessed Oct. 20, 2026).

[2] T. Lebo, S. Sahoo, and D. McGuinness. PROV-O: The PROV ontology. W3C Recommendation, April 2013. [Online]. Available: https://www.w3.org/TR/prov-o/.

[3] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel, J. Krishnamurthy, N. Lao, K. Mazaitis, T. Mohamed, N. Nakashole, E. Platanios, A. Ritter, M. Samadi, B. Settles, R. Wang, D. Wijaya, A. Gupta, X. Chen, A. Saparov, M. Greaves, and J. Welling. Never-ending learning. *Commun. ACM*, 61(5):103–115, May 2018. doi: 10.1145/3191513.

[4] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST '23)*, 2023. Also arXiv:2304.03442. doi: 10.1145/3586183.3606763.

[5] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956 [cs.CL], January 2025. [Online]. Available: https://arxiv.org/abs/2501.13956.

[6] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 2993–3000. IOS Press, October 2025. Also arXiv:2504.19413 [cs.CL]. doi: 10.3233/FAIA251160.

[7] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. arXiv preprint arXiv:2512.12818 [cs.CL], December 2025. [Online]. Available: https://arxiv.org/abs/2512.12818.

[8] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Trans. Assoc. Comput. Linguist.*, 12:157–173, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/.

[9] M. Zechner et al. pi: AI agent toolkit (unified LLM API, agent loop, TUI, coding agent CLI). GitHub repository, package @earendil-works/pi-coding-agent v1.0.2, 2026. [Online]. Available: https://github.com/earendil-works/pi (accessed Oct. 6, 2026).

[10] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating very long-term conversational memory of LLM agents. In *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, August 2024. preprint: arXiv:2402.17753. doi: 10.18653/v1/2024.acl-long.747. [Online]. Available: https://aclanthology.org/2024.acl-long.747/.

[11] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560, October 2023. rev. Feb. 2024. [Online]. Available: https://arxiv.org/abs/2310.08560.

[12] S. Daga, S. Sreedhar, and D. Shah. LongMemEval. Supermemory research report on LongMemEval-S (vendor report, not peer reviewed), May 2026. [Online]. Available: https://supermemory.ai/research/longmembench/ (accessed Oct. 6, 2026).

[13] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 86809–86836, 2025. Also arXiv:2410.10813. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html.

[14] PostgreSQL Global Development Group. PostgreSQL 17 documentation, chapter 12: Full text search (sections 12.2.2, 12.3.2, 12.3.3, 12.9) and section 64.4: GIN indexes. PostgreSQL 17.11 documentation, 2026. [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html (accessed Oct. 6, 2026).

[15] A. Kane et al. pgvector: Open-source vector similarity search for Postgres. GitHub repository, version 0.8.6, October 2026. [Online]. Available: https://github.com/pgvector/pgvector (accessed Oct. 6, 2026).

[16] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176 [cs.CL], June 2025. [Online]. Available: https://arxiv.org/abs/2506.05176.

[17] G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proc.*

32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09), pages 758–759, Boston, MA, USA, July 2009. doi: 10.1145/1571941.1572114.

[18] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 10088–10115, 2023. Also arXiv:2305.14314. doi: 10.52202/075280-0441.

[19] Neo4j, Inc. Query the database. in *Neo4j HTTP API Manual, Version 5*, 2026. [Online]. Available: `https://neo4j.com/docs/http-api/5/query/` (accessed Oct. 6, 2026).

[20] TypeSafe AI. SystemOne API: the Jev decision model. Web API, 2026. Model alias jev-latest. [Online]. Available: `https://api.typesafe.ai/v1/systemone` (accessed Oct. 21, 2026).

[21] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, NY, USA, 1993.

[22] J. Cohen. A coefficient of agreement for nominal scales. *Educ. Psychol. Meas.*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.

[23] LangChain. LangMem SDK: Long-term memory for agents. GitHub repository, 2025. [Online]. Available: `https://github.com/langchain-ai/langmem` (accessed Oct. 20, 2026).

[24] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. In *Proc. 39th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025. arXiv:2502.12110 [cs.CL].